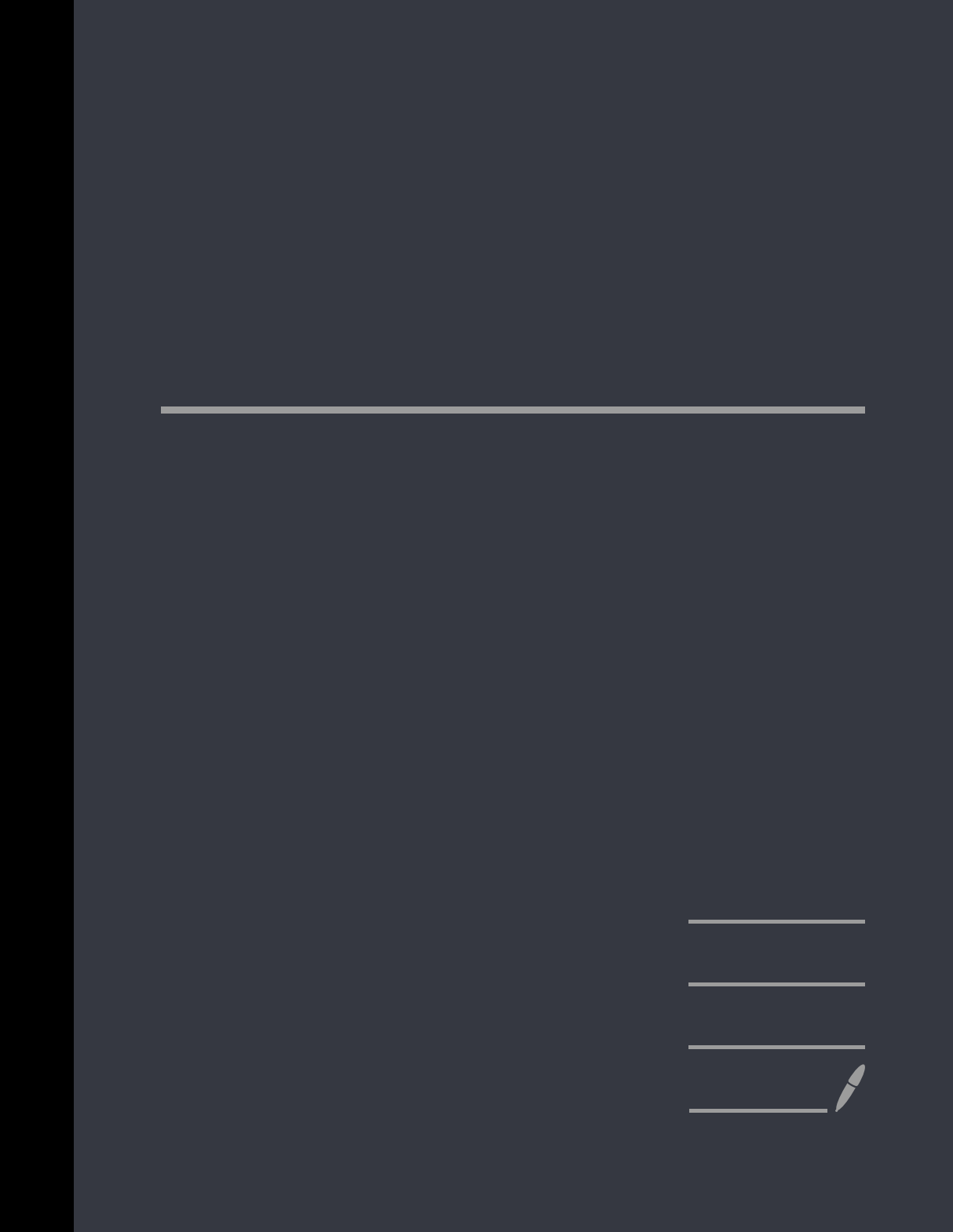


Cos'è il PWN-ing

Il PWNing consiste nell'exploitare la memory corruption.

Cos'è la memory corruption?

Corrompere la memoria in modo non atteso dal compilatore.
Controllando la memoria è possibile controllare il processo

Esempi pratici:
- Malware
- Attacchi a servizi remoti e applicazioni
- Sblocco dispositivi: Android Root, IOS jailbreak, gaming consoles

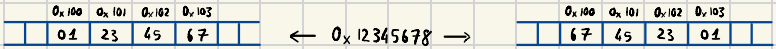
Vulnerabilità:
- Buffer Overflows: dati scritti in un buffer corrompono quelli negli indirizzi adiacenti al buffer
- Out-of-Bound Accesses: eccesso dei bound di un' array
- Format Strings: input string "evaluated" come comando
- Dangling Pointers: referenziare la memoria dopo che è stata liberata può far crashare il programma in esecuzione
- Type Confusion: codice senza type-checking

Ci son due tipi di sottoclassi di attacchi:
- Non-Control-Data: manipola lo stato dell'applicazione e i dati
- Control-Flow: manipola il flusso di esecuzione

Cos'è la memoria

La memoria è una sequenza di Bytes, in cui ogni Byte è identificato da un indirizzo.
Per la sua protezione alcune aree possono essere marcate come readable, writeable e executable.

Nella memoria non esistono i type, essi sono solo un'astrazione che indica come interpretare alcuni Byte.
Esistono 2 architetture di memoria:

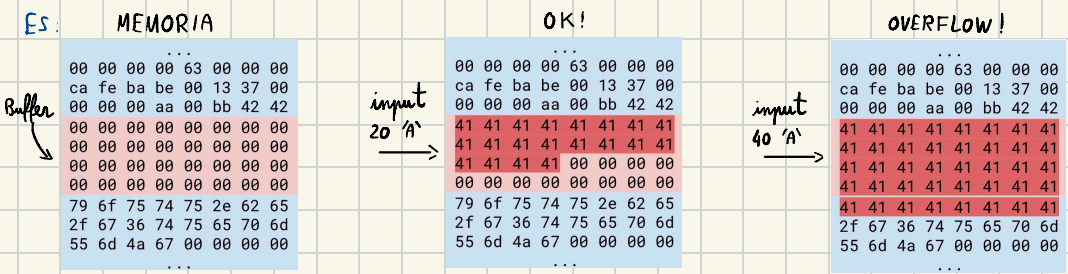


Big Endian
MSB salvato ad indirizzi più bassi

Little Endian
LSB salvato ad indirizzi più bassi

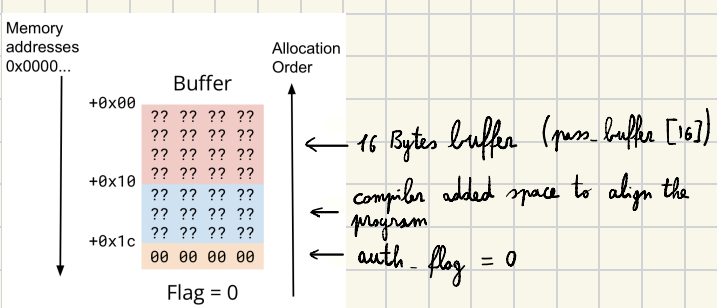
Gli array sono sequenze contigue di elementi, alcuni linguaggi (es. C/C++) non controllano gli array bounds.
Se non è il programmatore a controllare questi limiti, il programma potrebbe scrivere dei dati al di fuori dell'array

Es:

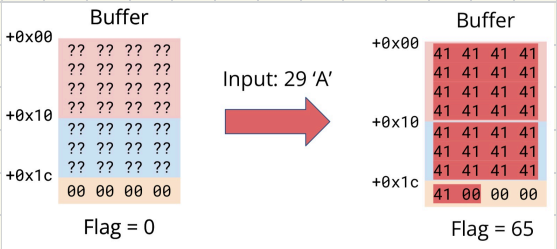


Es 1) Auth Overflow

```
int check_authentication () {
    int auth_flag = 0;
    char pass_buffer[16];
    printf ("Enter password");
    scanf ("%s", password_buffer);
    /* pass ok? => authflag = 1 */
    return auth_flag;
}
```



Se inserisco un valore con lunghezza > 16 Bytes nel password_buffer:



⇒ check_authentication() ritorna 65

⇒ if (check_authentication())
/* access granted */